

Reviewer Pack — Algebraic Geometry of Cubes and Circuit Monotone Width

Entry fce3718018f9 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-03 09:21:08 UTC

Algebraic Geometry of Cubes and Circuit Monotone Width

Entry ID: fce3718018f9 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-03 09:21:08 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Combinatorics (Cubes) **Field B** (complexity object): Boolean Circuit Complexity

Statement:

The minimal number of distinct cubes required to represent all possible output values in a circuit with monotone width k is linearly related to the circuit's monotone width, i.e., for every n -vertex circuit C with monotone width k , there exists a constant c such that the number of distinct cubes used to represent its outputs is $O(k^2 * \log(n))$.

Rationale (proposer's reasoning):

Cubes in algebraic combinatorics can be used to encode logical operations and circuits. By examining the minimal set of cubes needed to represent circuit outputs, we may uncover a connection between algebraic structure and circuit complexity that could shed light on the inherent difficulty of certain computational problems.

Taxonomy category: algebraic_cubes (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c8cef9fdf07a26cd

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across 30 random seeds, the number of distinct cubes required to represent all output values in a circuit with monotone width k is within $O(k^2 * \log(n))$ of k for each seed.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.90	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "algebraic combinatorics" AND "cubes" AND "Boolean circuit complexity"
- "monotone width" AND "cube representation" AND "circuit complexity" - "linear relationship"
AND "distinct cubes" AND "monotone width k"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def evaluate_circuit(circuit, input_values):
        stack = []
        for gate in circuit:
            if gate[0] == 'NOT':
                stack.append(not input_values[gate[1]])
            elif gate[0] == 'AND':
                stack.append(stack.pop() and input_values[gate[1]])
            elif gate[0] == 'OR':
                stack.append(stack.pop() or input_values[gate[1]])
        return stack[-1]

    def find_min_cubes(circuit):
        n = len(circuit)
        output_values = set()
        for i in range(2**n):
            input_values = {j: (i >> j) & 1 for j in range(n)}
            output_values.add(evaluate_circuit(circuit, input_values))

        cubes = []
        for value in output_values:
            cube = [0] * n
            for i in range(n):
                if evaluate_circuit(circuit, {j: (value >> j) & 1 for j in range(i)}) != evaluate_
                    cube[i] = 1
            cubes.append(cube)
```

```

        return len(cubes)

n_max = 40
instances_tested = 30
total_metric_value = 0.0
conjecture_holds = True
counterexample = ""

for _ in range(instances_tested):
    k = random.randint(5, 40)
    circuit = []
    for i in range(k):
        gate_type = random.choice(['NOT', 'AND', 'OR'])
        if gate_type == 'NOT':
            input_index = random.randint(0, len(circuit) - 1)
            circuit.append(('NOT', input_index))
        else:
            input_indices = random.sample(range(len(circuit)), 2)
            circuit.append((gate_type, input_indices[0], input_indices[1]))

    num_cubes = find_min_cubes(circuit)
    expected_bound = k**2 * math.log(n_max)

    if abs(num_cubes - expected_bound) > 1e-6:
        conjecture_holds = False
        counterexample = f"n={len(circuit)}, k={k}, num_cubes={num_cubes}, expected_bound={exp

metric_value = total_metric_value / instances_tested

return {
    "metric_name": "Number of distinct cubes",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37, 41, 43,

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_metric_value = sum(r["metric_value"] for r in results) / len(results)
    std_metric_value = math.sqrt(sum((r["metric_value"] - mean_metric_value)**2 for r in results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fraction

```

```

else:
    first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
    print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\", first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4ebd83a8.py", line 91, in <module>
    result = run_trial(seed)
              ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_4ebd83a8.py", line 64, in run_trial
    input_indices = random.sample(range(len(circuit)), 2)
                      ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/usr/lib/python3.12/random.py", line 430, in sample
    raise ValueError("Sample larger than population or is negative")
ValueError: Sample larger than population or is negative

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevented the production of data necessary to evaluate the conjecture. | next: Review and debug the test code to ensure it can run successfully and produce the required data.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	26036
2	preregistration	ollama_remote	glm4:latest	0	0	10547
3	novelty	ollama_remote	glm4:latest	0	0	8592
4	novelty	ollama_remote	glm4:latest	0	0	13045
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14836
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12653
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17993
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16504
9	judge	ollama_remote	glm4:latest	0	0	21222

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 141427 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in research/audit/fce3718018f9.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/fce3718018f9.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/fce3718018f9.tar.gz (if generated)